

PHP OOP Exercise: Music Store Guitars

In this exercise, you will practice working with **constructors**, **object cloning**, and the **null coalescing operator** in PHP. You will create and test a simple `Guitar` class step by step.

Create a folder outside of your `web250` folder. Name the folder `in-class-constructors`. Add the following files to the folder. Also provide an example of applying a data type (type hint).

Example of type hinting:

```
php public function __construct(string $brand, string $model, float $price) { // assign values here }
```

Step 1 — Create a class with a constructor

1. Inside your `in-class-constructors` folder, create a new file called `step1_guitar.php`.
2. Define a class named `Guitar`.
3. Add three properties: `$brand`, `$model`, and `$price`.
 - Use the type hints `string` for brand and model, `float` for price.
 - For now, you can make them `public`.
4. Write a constructor method (`__construct`) that accepts three parameters: brand, model, and price.
 - Inside the constructor, assign the values to the class properties using `$this->propertyName = $parameterName;`.
5. Add a method named `summary()` that returns a string in this format:
`Brand Model - $Price`
Tip: Use `number_format($this->price, 2)` to format the price with two decimals.
6. In your main program, create two `Guitar` objects with real brands and models, and then echo the result of `summary()` for each.

Step 2 — Clone an object

1. In the same folder, create a new file called `step2_clone.php`.
2. Copy your `Guitar` class into this file.
3. Create one `Guitar` object (for example, an Ibanez model).
4. Use the PHP `clone` keyword to make a copy of the object.
 - Example:

```
php $clone = clone $original;
```
5. Change one or more properties on the clone (e.g., update the model name and price).
6. Echo `summary()` for both the original and the clone. Notice how the two objects are separate.

Hint: After cloning, changing `$clone->model` will not affect `$original->model`.

Step 3 — Use an array in the constructor with null coalescing

1. In the same folder, create a new file called `step3_array_constructor.php`.
2. Rewrite your `Guitar` class so the constructor accepts a single array parameter instead of three separate values.
 - Call this parameter `$args`.
 - Example:

```
php public function __construct(array $args = []) { $this->brand = $args['brand'] ?? null; // continue with model and price }
```
3. Use the **null coalescing operator** (`??`) so that if a key is missing in the array, the property will default to `null`.
4. Update your properties so they allow `null` values (`?string`, `?float`).
5. In your `summary()` method, check for `null` values and display a fallback message:
 - If brand or model is null → display `"Unknown Brand"` or `"Unknown Model"`.

- If price is null → display "Price not available" .

6. Test your constructor with three objects:

- One with all fields filled in.
- One missing the price.
- One created with an empty array.

Hint: Start by writing the constructor to pull values from `$args['brand'] ?? null` , `$args['model'] ?? null` , and `$args['price'] ?? null` .

////////////////////////////////////

Expected results

By the end, your program output might look something like this:

```
Fender Player Telecaster - $799.99
Gibson Les Paul Studio - $1,699.00
Original: Ibanez RG450DX - $499.00
Clone: Ibanez RG550 - $899.00
Fender Jazzmaster - $1,199.00
Epiphone SG Special - Price not available
Unknown Brand Unknown Model - Price not available
```